

[git@github.com:mikeknapp/candy-machine.git](https://github.com/mikeknapp/candy-machine.git)

[git@github.com:AUTOMATIC1111/stable-diffusion-webui.git](https://github.com/AUTOMATIC1111/stable-diffusion-webui.git)

[git@github.com:mudler/LocalAI.git](https://github.com/mudler/LocalAI.git)

[git@github.com:StanfordSNR/puffer.git](https://github.com/StanfordSNR/puffer.git)
<https://github.com/bookieandco/crispy-waddle>

Repository Asset Integration Plan

Objective

Evaluate these repositories as reusable infrastructure for the Jhadina ecosystem.

Do **not** treat them as standalone applications.

Treat each repository as a capability that can be wrapped behind an adapter and exposed through DELIA or MARISA without violating the existing architecture.

The existing architecture remains the source of truth:

- JANET = reasoning, memory, pattern engine
- DELIA = engineering and technical execution
- MARISA = operational workflow execution

No duplicate memory systems.

No duplicate project databases.

No duplicate asset libraries.

External repositories are capability providers only.

Repository 1

Candy Machine

Repository:

git@github.com:mikeknapp/candy-machine.git

Purpose:

Evaluate whether this repository contains reusable automation or asset generation components.

Do not copy the application wholesale.

Questions to answer:

- What reusable modules exist?
- Which components can become adapters?
- Which parts should remain external services?
- Which parts duplicate existing Jhadina functionality?

Deliverable:

A capability report, not an implementation.

Repository 2

Stable Diffusion WebUI

Repository:

git@github.com:AUTOMATIC1111/stable-diffusion-webui.git

Role:

Primary local image-generation backend.

This should **not** become the Film Studio.

Instead:

JANET

↓

Creative Intent

↓

DELIA



Image Generation Adapter



Stable Diffusion WebUI



Generated Assets



Existing Asset Library

Use cases:

- concept art
- character sheets
- environment generation
- prop generation
- storyboard frames

The generated images should be imported into the existing asset system with metadata.

Do not create a second image library.

Repository 3

Puffer

Repository:

`git@github.com:StanfordSNR/puffer.git`

Treat this as research infrastructure.

Determine whether any ideas or components are useful for:

- media delivery
- adaptive streaming
- video quality optimization
- remote viewing
- production workflows

Only integrate pieces that solve an existing problem.

Do not integrate simply because the repository exists.

Produce:

- architectural assessment
 - possible integration points
 - reasons to reject any unnecessary components
-

Repository 4

LocalAI

Repository:

`git@github.com:mudler/LocalAI.git`

Role:

Local inference gateway.

Architecture:

JANET

↓

DELIA

↓

Inference Adapter

↓

LocalAI

↓

Local Models

Possible uses:

- local LLM inference
- embeddings
- speech
- image generation
- offline development

The adapter should hide implementation details.

JANET should not know whether inference comes from:

- OpenAI
- Anthropic
- LocalAI
- another provider

Inference providers should be interchangeable.

Existing Repository

<https://github.com/bookieandco/crispy-waddle>

This repository should become the integration point rather than a collection of unrelated tools.

The goal is to evolve it into the orchestration layer for the entire Jhadina ecosystem rather than embedding external projects directly. (The linked repository could not be verified from this environment, so inspect its current structure before deciding where each capability belongs.)

([Beat the Bookie](#))

Suggested structure:

crispy-waddle/

adapters/

localai/
stable_diffusion/
candy_machine/
puffer/

core/
janet/
delia/
marisa/

assets/

reasoning/

memory/

film_studio/

world_builder/

Integration Rules

Every external repository must be evaluated using the same process.

Step 1

Audit the repository.

Determine:

- purpose
 - maturity
 - dependencies
 - license
 - maintenance status
 - security concerns
-

Step 2

Identify reusable capabilities.

Do not integrate entire applications if only a small subsystem is needed.

Step 3

Create an adapter.

The adapter is the only layer that communicates with the external project.

No business logic should depend directly on third-party code.

Step 4

Register the capability.

Record:

- repository
 - version
 - license
 - owner
 - integration status
 - limitations
 - known risks
-

Step 5

Verification

Before marking any capability complete:

- adapter tests pass
 - existing regression tests pass
 - no duplicate functionality is introduced
 - architecture boundaries remain intact
-

Long-Term Goal

The repositories should work together like interchangeable capability providers.

JHADINA

|

JANET

|

DELIA

Image Adapter

LocalAI Adapter

Streaming Adapter

Automation Adapter

Stable Diffusion WebUI

LocalAI

Puffer

Candy Machine

|

Existing Asset Library

Existing Memory System

Existing Reasoning System

The objective is not to merge projects.

The objective is to build a clean adapter layer so external repositories can provide capabilities while Jhadina remains the system that owns memory, reasoning, assets, workflows, and long-term orchestration.

Jhadina Asset Preparation Plan

Objective

Prepare structured assets that can be used across:

- JANET reasoning experiments
- World Builder
- Film Studio expansion
- Creative memory
- Future Decision Engine evaluation

The goal is not just collecting files.

The goal is creating reusable, searchable knowledge objects that the system can reason over.

Priority 1 — Reasoning Experiment Assets

Prepare real-world decision examples before building automated decision systems.

Create 5–10 reasoning scenarios.

Format:

Scenario:

Question:

Context:

Available evidence:

Constraints:

Possible options:

Final decision:

Outcome:

Example: OverageOS-Style Scenario

Scenario:

County surplus funds opportunity

Question:

Should this opportunity be pursued?

Context:

A county has potential surplus tax foreclosure funds available.

Evidence:

- estimated recoverable amount
- county requirements
- property records
- claimant difficulty

Constraints:

- legal risk
- cost
- time required

Options:

- pursue
- reject
- monitor

Decision:

Outcome:

Example: Business Scenario

Scenario:

PupsonStuff product decision

Question:

Should resources go toward advertising or creating more products?

Evidence:

- current sales
- margins
- traffic

- conversion rate

Constraints:

- available budget
- production capacity

Options:

Decision:

Outcome:

These become reasoning_events and later reasoning_reviews.

Priority 2 — World Builder Assets

Prepare structured locations.

For each location create:

Name:

Description:

Era:

Mood:

Architecture:

Colors:

Sounds:

Weather:

Stories associated:

Visual references:

Example

Name:

The Block

Description:

Urban neighborhood centered around local businesses and community spaces.

Mood:

Late summer evening.

Architecture:

Brick buildings,
corner stores,
street murals.

Colors:

Warm street lighting.

Sounds:

Cars passing,
basketball courts,
music from windows.

Weather:

Hot summer night.

Stories:

Character meeting locations.

Reference Image Metadata

Do not store images without context.

Bad:

warehouse.jpg

Good:

industrial_warehouse_night_reference

Purpose:
Warehouse scene

Traits:
Cold lighting,
large open space,
rusty metal,
dramatic shadows

Priority 3 — Character Assets

Create a character bible for every recurring character.

Format:

Name:

Age:

Physical traits:

Height:

Build:

Clothing:

Personality:

Speech style:

Movement style:

Likes:

Dislikes:

Never changes:

The “Never changes” section becomes locked

Traits.

Character Reference Pack

Prepare:

- front view
- side view
- expressions
- clothing variations
- important props
- environment references

Purpose:

Maintain consistency across future creative generation.

Priority 4 — Film Studio Assets

Create a shot library.

Format:

Shot Type:

Purpose:

Camera:

Movement:

Framing:

Lighting:

Mood:

Example:

Example

Shot Type:

Slow push-in

Purpose:
Build tension.

Camera:
35mm lens.

Movement:
Dolly forward.

Framing:
Medium close-up.

Lighting:
Low-key blue lighting.

Mood:
Suspense.

Priority 5 — Visual Style Library

Create reusable style definitions.

Format:

Style Name:

Lighting:

Camera:

Composition:

Color:

Mood:

References:

Example

Style Name:

Neo Noir

Lighting:

High contrast.

Camera:

Wide cinematic lenses.

Composition:

Deep shadows.

Color:

Dark urban tones.

Mood:

Mystery and tension.

Priority 6 — Music Restoration Assets

Prepare documented restoration cases.

For each song:

Song:

Year:

Source quality:

Known problems:

Restoration goal:

Previous decisions:

Final evaluation:

Example

Song:
Party Nites

Source:
Compressed digital file.

Problems:
Muddy low end.
Harsh cymbals.

Goal:
Improve clarity while preserving character.

Previous decisions:
Controlled high frequency harshness.
Investigated kick reinforcement.

Final evaluation:

Priority 7 — Universal Asset Metadata

All assets should eventually follow a shared format.

Example:

```
{  
  "name": "",  
  "type": "",  
  "project": "",  
  "purpose": "",  
  "created_by": "",  
  "references": "",  
  "approved": false,  
  "notes": ""  
}
```

Recommended Repository Structure

/assets

/characters

/locations

/worlds

/styles

/shots

/reasoning_examples

/music_cases

Recommended Preparation Order

Phase 1

Create:

1. Five reasoning scenarios
 2. Two or three character bibles
 3. Ten location/world references
-

Phase 2

Create:

4. Shot library
 5. Visual style library
 6. Music restoration case documentation
-

Guiding Principle

Assets should not just be stored.

They should be structured so JANET can:

- compare them
- evaluate them
- learn patterns from them
- connect them across projects

The goal is to build a knowledge foundation, not a file collection.